WEEK 1
Team Based Learning
iRAT activities

Questions with wrong answers: NONE

WEEK 2
Git
iRAT activities

Questions with wrong answers: NONE

Important notes
Git commands

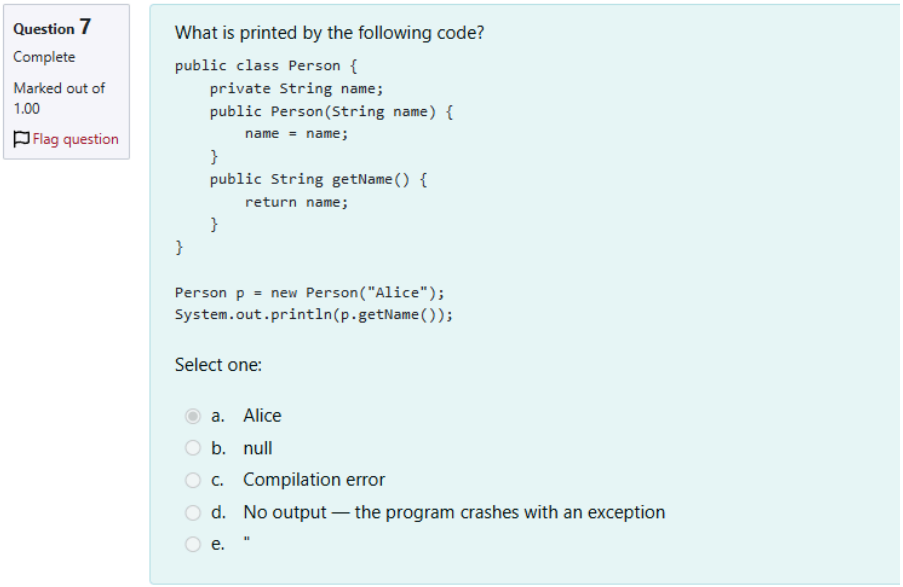
Commands	Functions
cd “folder_name”	Move from one folder to another
wq	Save your changes and quit
w	Save the file
q	Quit Vim
q!	Quit without saving
i	Enter Insert Mode
Esc	Leave Insert move and go back to Command mode
git clone	Create a local copy of an existing remote Git repository
git add	Saves changes from your working directory into Git staging area (index)
git commit	Captures a snapshot of your project’s currently staged changes and saves it permanently to your local repository history
git push	Command uploads local repository commits to a remote server
git pull	Downloads changes from a remote repository and immediately integrates them into your current local branch
git init	Creates a new, empty Git repository or reinitialises an existing one
git fetch	Downloads commits, files, and references from a remote repository into your local repository without merging them into your active local branches

git log	Displays a repository's complete commit history in reverse chronological order
git status	Displays the current state of your working directory and the staging area, showing which changes are tracked, untacked, or ready to be committed
git fork	A server-side, independent copy of a repository hosted under your own account on platforms (But you can't edit it, it's just a copy)

WEEK 3
Classes and Objects

iRAT activities

Questions with wrong answers



Question 7
Complete
Marked out of 1.00
Flag question

What is printed by the following code?

```
public class Person {
    private String name;
    public Person(String name) {
        name = name;
    }
    public String getName() {
        return name;
    }
}

Person p = new Person("Alice");
System.out.println(p.getName());
```

Select one:

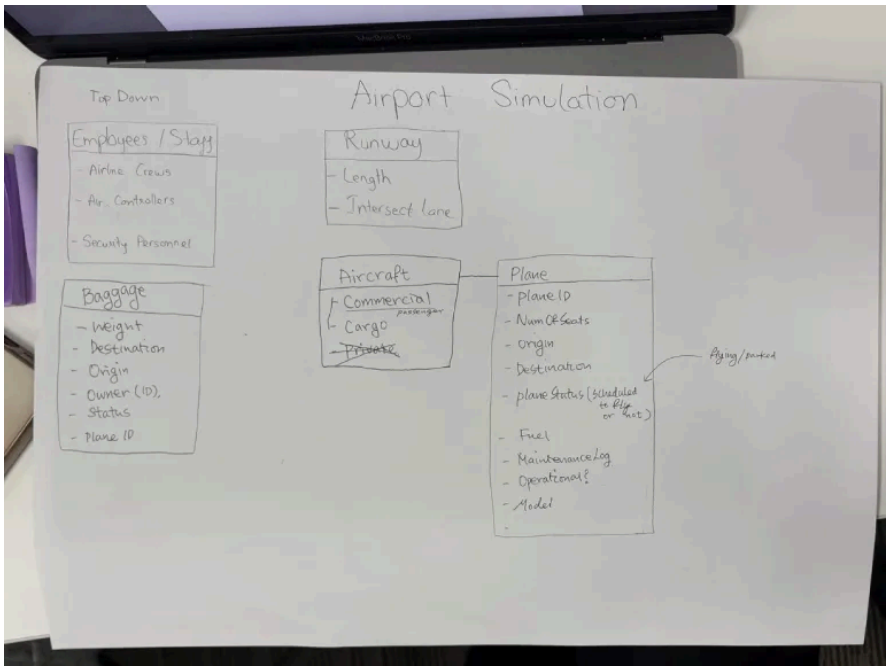
a. Alice
b. null
c. Compilation error
d. No output — the program crashes with an exception
e. ''

Correct answers: B: null
Reason: Because there is no "this.name=name" therefore java would print out "null" as "name=name"

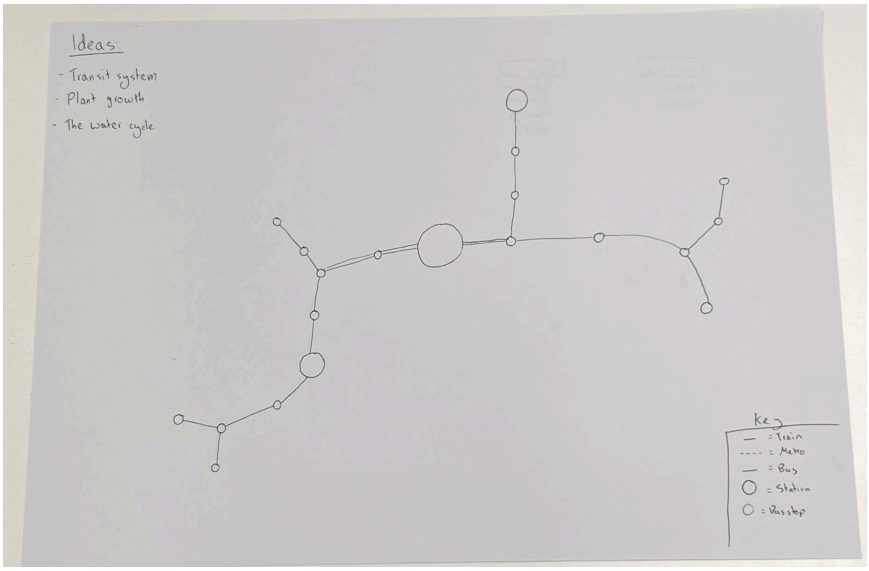
Project

Draw.io link:
https://app.diagrams.net/#G1_CHOv7-_QWzYlbBotdgZEz5XpHFofoiV#%7B%22pageId%22%3A%22Vp9QY6zV6l614nxmznQm%22%7D

Our group's ideas of simulation: Airport simulation



Other group's ideas simulation: Traffic simulation



WEEK 4
Inheritance and Overloading
iRAT activities

Questions with wrong answers

Question 8
Complete
Marked out of 1.00
Flag question

Given the following code, which statement is true about the constructor call `super()`?

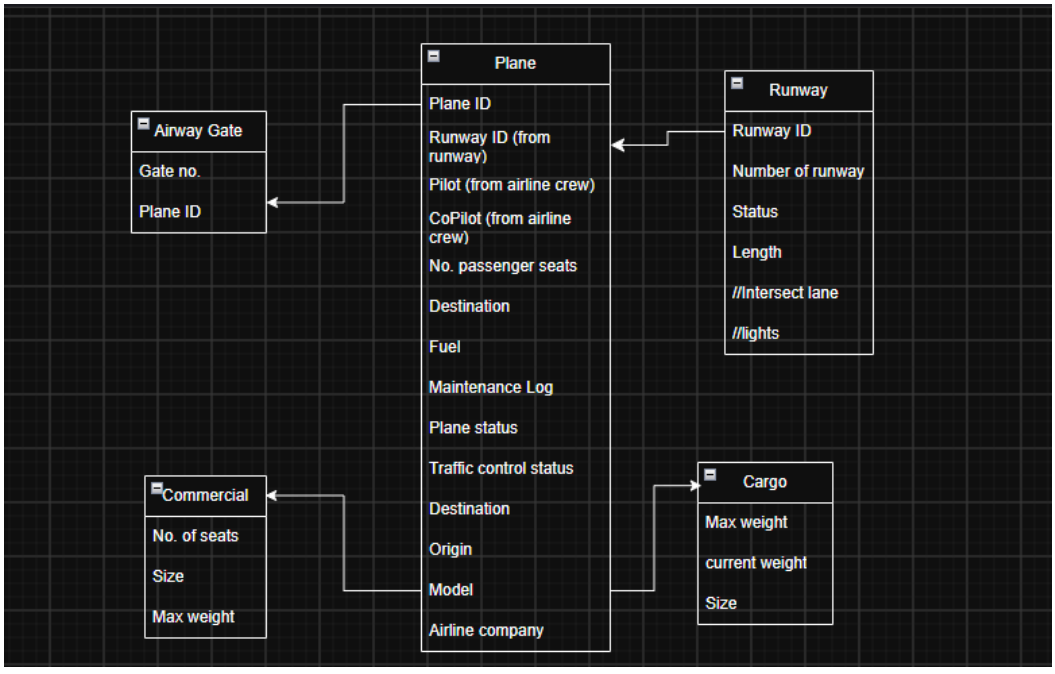
```
class Animal {
    String name;
    Animal() {
        name = "unknown";
    }
}
class Dog extends Animal {
    String breed;
    Dog(String breed) {
        this.breed = breed;
    }
}
```

Select one:

- a. Dog's constructor cannot call `super()` because Dog has its own fields.
- b. The code does not compile because Dog must explicitly call `super()`.
- c. The compiler implicitly inserts `super()` as the first statement in Dog's constructor.
- d. The Animal() constructor is not called because Dog does not reference name.
- e. `super()` must be the last statement in Dog's constructor to ensure breed is initialised first.

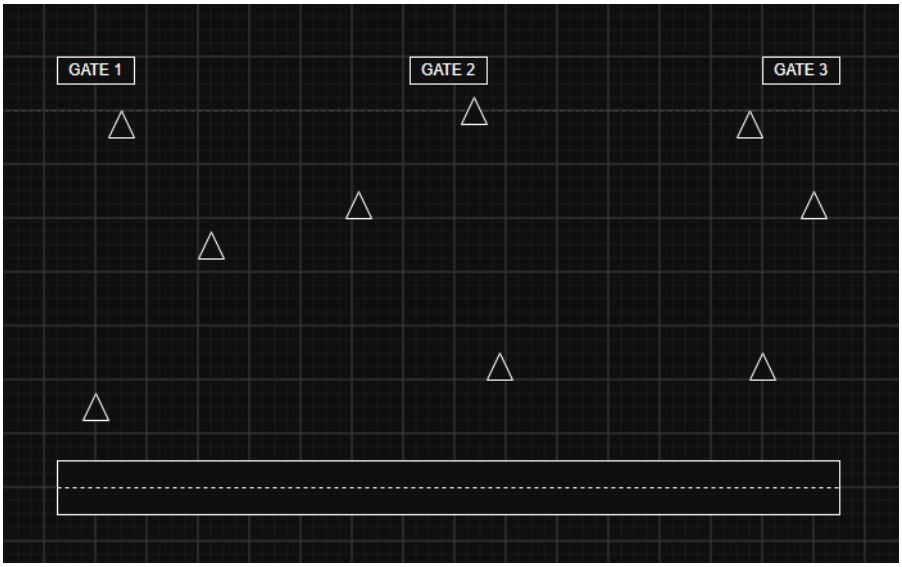
Correct answers: C
Reason:

Our group's ideas of simulation: Airport Traffic Simulation



- We have decided to focus on the plane and runway classes for now as it helps the most with our simulation.
- Compared to last week where we have lots of classes and entities, we removed baggage, staff, aircrews and passengers out of it.
- For now we could have 1 run way, 3 gates and a few planes (triangles) in the panel to move around. For example:

My Ideas for later simulation



- Maybe for later weeks, we could develop a simulation more on passengers getting on the plane once the plane stops.
- Or a bus that drives passengers to the plane on a bus lane.

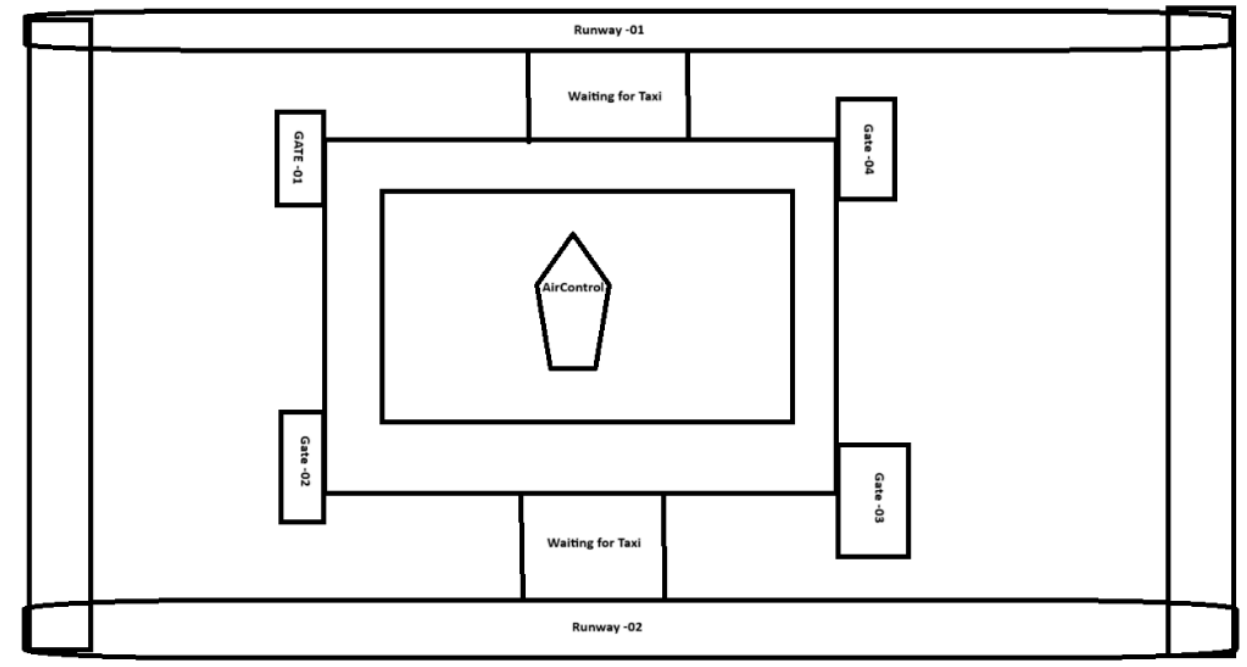
WEEK 5
Generics
iRAT activities

Questions with wrong answers

Correct answers:
Reason:

Project

Airport sketch



- Start putting on setters and getters

WEEK 6
Exceptions
iRAT activities

Questions with wrong answers: None

Important notes

Try and Catch:

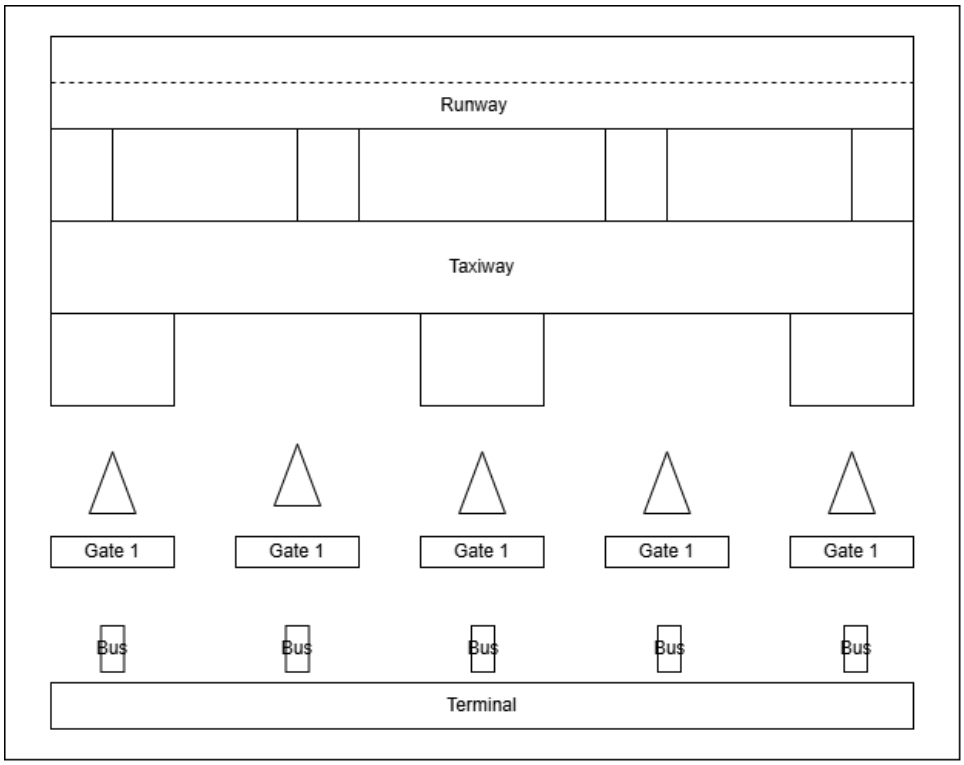
- Basically "run this program and try this, if it doesn't work, catch this exception without crashing the program."
- Useful because: things can go wrong while a program runs, and you want your program to handle them sensibly.
    - For example: "Enter your age:" but users enter a String "hello", the program will then run "please enter valid age" instead of crashing.

Finally block:

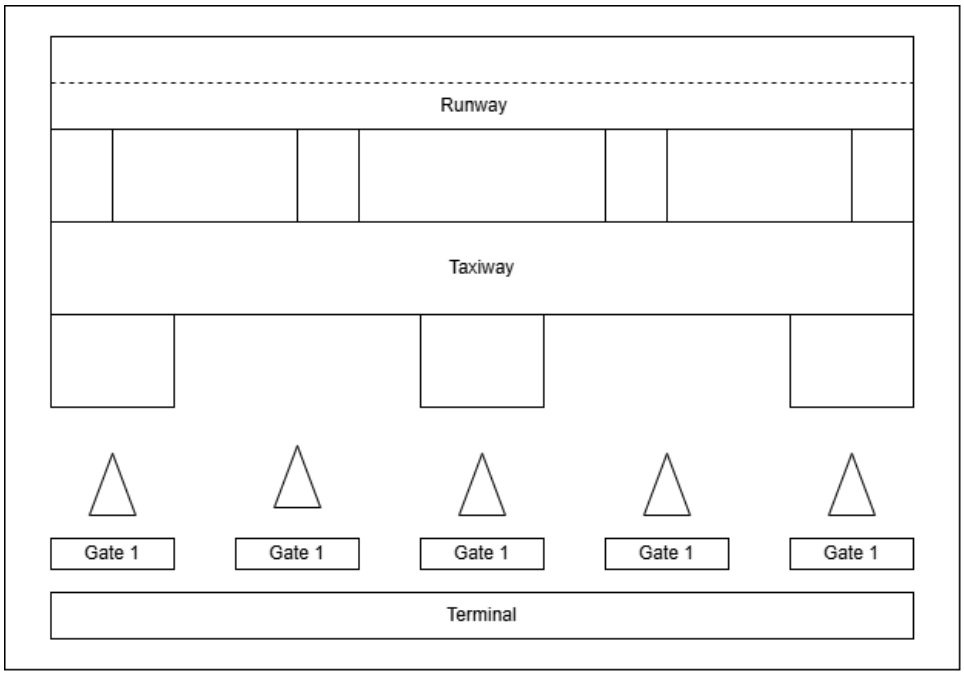
- Run whether an exception happens or not.
- Useful because: finally block still runs even if there's an error.

NullPointerException	Unchecked exception	No
IOException	Checked exception	Yes
ArrayIndexOutOfBoundsException	Unchecked exception	No
IllegalArgumentException	Unchecked exception	No
StackOverflowError	Unchecked exception	No

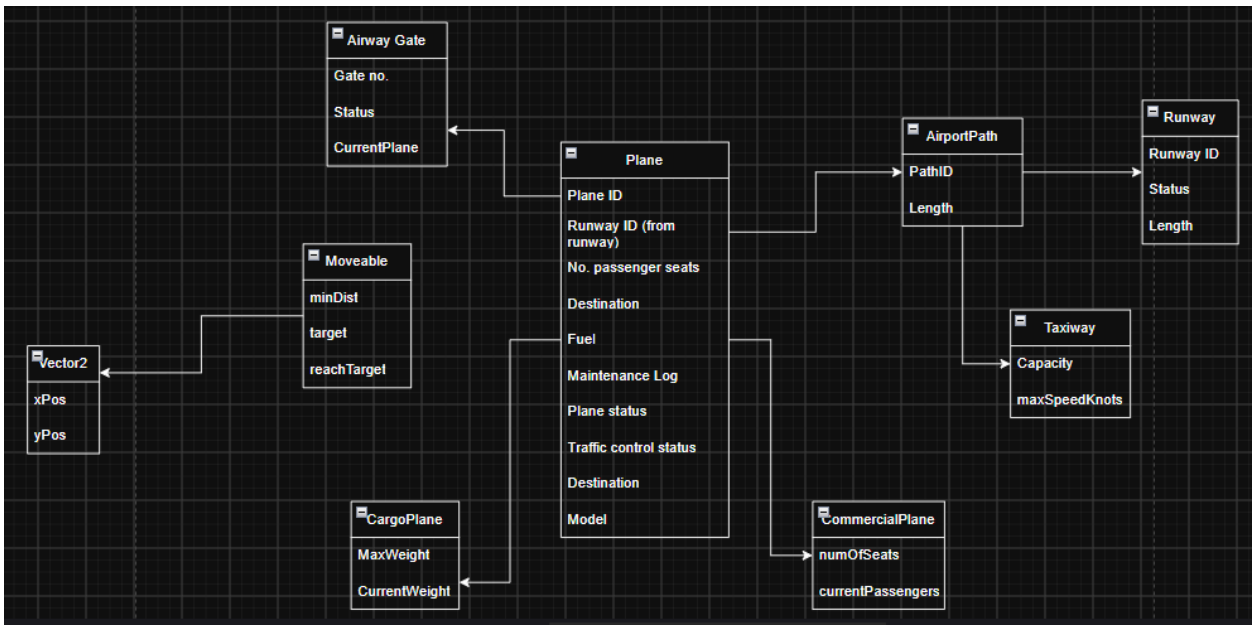
My ideas for later simulation



My Group's ideas for later simulation



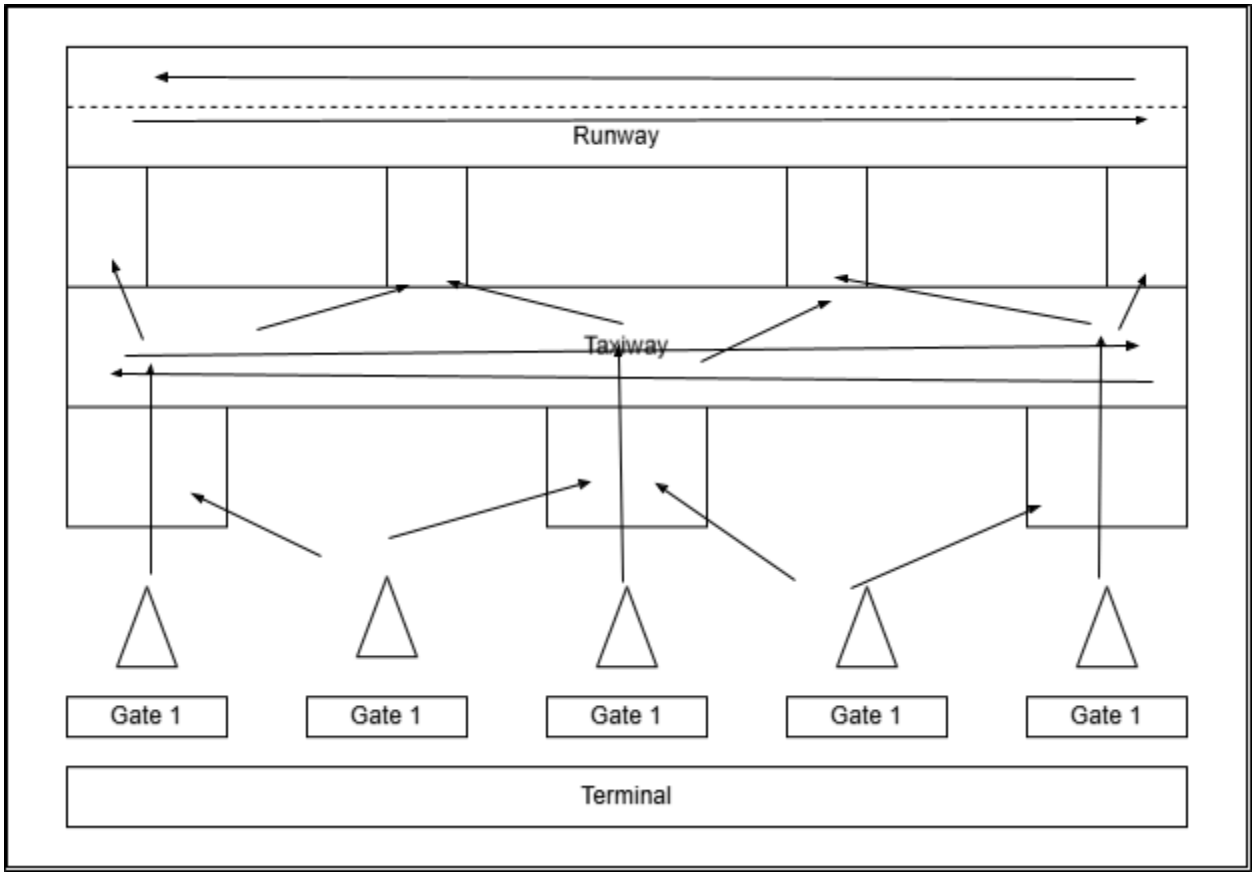
UML design



WEEK 7
Patterns 1
PRESENTATION

Presentation Feedback:

- Organise our files
- Sort out a way to spawn planes
- Develop better coloring design for our panel



WEEK 8
Patterns 2
Debate and best zingers

Debate topics: Should a simulation prioritise realism or simplicity?

My team's position: A simulation should have a balance between realism with enough complexity management.

Best zingers: A simulation that perfectly models reality but cannot run reliably is not realistic, but rather unfinished.

Why is it effective: This supports our decision to simplify airport traffic by using nodes and allowing only one plane on the runway. It showed that a smaller, reliable system can demonstrate the intended behaviour better than an overly complicated and incomplete one.

WEEK 9
Behaviour Parameterisation

Project update

```
public void createRoutes() {
    routeGate1 = new Route();
    routeGate1.addNode(gateNode);
    routeGate1.addNode(lowerPathNode);
    routeGate1.addNode(taxiwayNode);
    routeGate1.addNode(upperPathNode);
    routeGate1.addNode(runwayEntrance);
    routeGate1.addNode(runwayExit);

    routeGate2 = new Route();
    routeGate2.addNode(gateNode2);
    routeGate2.addNode(lowerPathNode2);
    routeGate2.addNode(taxiwayNode);
    routeGate2.addNode(upperPathNode2);
    routeGate2.addNode(runwayEntrance);
    routeGate2.addNode(runwayExit);
}
```

```
public Airport() {
    planes = new ArrayList<>();

    createNodes();
    createRoutes();
    connectNodes();

    runway = new Runway (pathID: "Runway 1",
}
```

- Changing plane route update. Instead of creating a specific route for each plane, I changed it to BFS route finding which is an algorithm used to find a path through connected nodes.

```
private Route departureRoute;
private Route arrivalRoute;
private Route routeGate1;
private Route routeGate2;
```

- Since I've used BFS route findings, route nodes are not necessary anymore and its removal would be essential.

WEEK 10
Streams

Original loop:

```
List<Plane> boardPlanes = new ArrayList<>();

for (Plane plane : airport.getPlanes()) {
   if (!plane.isFinished()) {
      boardPlanes.add(plane);
   }
}

boardPlanes.sort(
   (plane1, plane2) ->
      plane1.getPlaneID().compareTo(plane2.getPlaneID())
);
```

Stream replacement:

```
List<Plane> boardPlanes = airport.getPlanes()
   .stream()
   .filter(plane -> !plane.isFinished())
   .sorted(
      (plane1, plane2) ->
         plane1.getPlaneID().compareTo(
            plane2.getPlaneID()
         )
   )
   .toList();
```

- The stream pipeline is easier to read because it expresses the operations in order:
    - .stream(): retrieve the planes
    - .filter(): finished planes
    - .sorted(): sort the remaining planes
    - .toList(): create a list with the results
- It avoids manually creating a list and adding planes inside a loop. Both versions are correct and do not modify the original plane list.
- For the small number of planes in the simulation, the performance difference is insignificant. The stream may have a small amount of additional overhead, but its concise and declarative structure makes the purpose of the code clearer.

WEEK 11
Collecting

WEEK 12
Parallelism

- My current project doesn't use parallel but theoretically it could. For a small number of planes in this simulation, I feel it is unnecessary to use parallel streams. So it is kept as a normal sequential stream like:

  airport.getPlanes().stream()

- A possible parallelism opportunity was filtering and sorting the planes displayed on the flight board. This could be implemented by replacing .stream() with .parallelStream(), allowing different planes to be processed concurrently. However, I kept the sequential stream because the simulation only contains a small number of planes. The overhead of dividing and combining parallel tasks would probably be greater than any performance improvement.
- Parallelism would also introduce synchronisation challenges because planes share mutable resources, including nodes, routes, the runway, and the airport's plane list. If multiple threads attempted to reserve the same node or enter the runway simultaneously, race conditions could occur.
- The graphical interface also uses Swing, which expects interface updates such as repaint() to be handled through its Event Dispatch Thread. A sequential stream therefore made the simulation simpler, predictable, and safer while still providing sufficient performance.

WEEK 13
Student Showcase
Presentation